# CAMO: A High-Performance CIM-based Lightweight CNN Accelerator for Mobile Devices

Xin Ju[1], Renyu Yang[1], Mei Wen[1]✉, Junzhong Shen[1], Tianyu Wang[2], Zhaoyan Shen[3], Zili Shao[4] and Bin Liang[1]
[1]Key Laboratory of Advanced Microprocessor Chips and Systems, National University of Defense Technology, Changsha, China
[2]College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China
[3]School of Computer Science and Technology, Shandong University, Shandong, China
[4]Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong, China
✉Corresponding author: Mei Wen, Email: meiwen@nudt.edu.cn

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11043589

***Abstract*—Digital Compute-in-Memory (CIM) macros revolutionize the Von Neumann architecture by significantly reducing data movements between CPU and memory. However, when dealing with lightweight CNNs with various convolution types, existing GEMM (general matrix multiplication)-oriented solutions suffer from underutilization and large activation traffic, leading to unsatisfied energy and area efficiencies. In this context, we propose *CAMO*, in which the key contributions are: (1) A novel convolution mapping mechanism suitable for CIM macros, that maximizes data reuse and reduces activation traffic. (2) A convolution-capable CIM macro, that also supports small-scale GEMM. (3) A CIM-based architecture that supports multiple computing modes. The experimental results show that *CAMO* achieves up to 31.49× performance speedup and 77.5% activation traffic reduction compared to the baseline architecture.**

## I. Introduction

With reduced computation complexity and parameters, lightweight convolution neural networks (CNNs) provide promising solutions for CNN inference on mobile devices. However, the widely-used depthwise separable convolution in lightweight CNNs [1]–[3] which has various convolution patterns poses a lot of challenges for general hardware architectures [4]–[7]. For instance, there are three types of convolution in MobileNetV2 [1], a prominent lightweight CNN model, i.e., (1) standard convolution (STConv), (2) depthwise convolution (DWConv), and (3) pointwise convolution (PWConv). It is of vital importance to explore a novel architecture to achieve both high throughput and energy efficiency with a low area cost.

Compute-in-memory architecture exhibits great potential to achieve high energy efficiency as it combines compute logic with memory [8]. Some analog CIM macros [9], [10] support DWConv, but their accuracy is susceptible to voltage and temperature. Digital CIM macros focus on specific operations, such as simultaneous multiply accumulation (MAC) and weight updates [11], variable precision support [12], and configurable adder tree [13], these approaches cannot fit lightweight CNNs. To provide a better institution, Figure 1 shows the layer-wise MAC utilization in processing MobileNetV2 with state-of-the-art (SOTA) digital CIM macro [11]. Firstly, converting STConv and DWConv to GEMM [14] will not only bring additional on-chip storage and redundant data movements, but also has the following issues for DW-

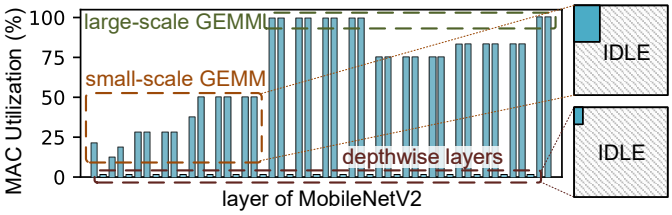



**Fig. 1.** Layer-wise utilization of MobileNetV2 on state-of-the-art digital CIM macro.

Conv: weights are converted to vectors, which means the input feature maps (Ifmaps) are non-reusable, and the accumulation length is short (i.e., 9), thus MAC utilization is extremely low. Secondly, for the PWConv, although it is GEMM in essence, the scale of most PWConv layers is small, also leading to a sub-optimal MAC utilization.

To address the above issues, we propose *CAMO*, a high-performance accelerator based on a convolution-capable CIM macro (C-CIM) for lightweight CNNs, supporting different compute mode switches for various convolution types. Our evaluation results show that *CAMO* achieves a 31.49× increase in MAC utilization and a 77.5% reduction in activation traffic compared to the baseline.

We make the following contributions in this paper.

- We propose a novel convolution mapping mechanism suitable for CIM macro, COMAS, aimed at maximizing data reuse and minimizing activation traffic.
- We propose a convolution-capable CIM macro, C-CIM, featuring a configurable adder tree to facilitate small-scale GEMM.
- We propose a CIM-based architecture, *CAMO*, that supports multiple computing modes to accommodate different types of convolution in lightweight CNNs.

## II. Background and Motivation

In digital CIM macro, data is depicted as the result of both a logic gate array and an adder tree, the multiplication of the logic gate array yields a partial product, while the adder tree facilitates accumulation to implement the MAC operation [8]. As shown in Figure 3, a traditional digital CIM macro consists of $V$ subarrays and each subarray contains an SRAM, a logic gate array, and an adder tree. The size of the SRAM is $Row \times$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:06 UTC from IEEE Xplore. Restrictions apply.

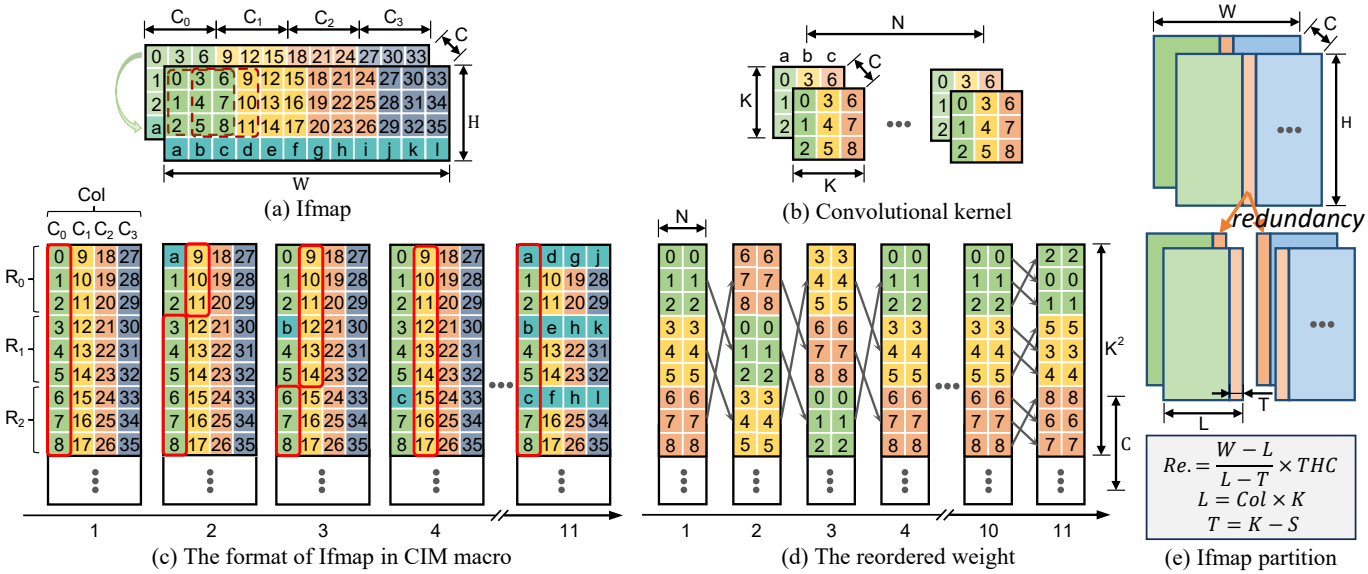



**Fig. 2.** An example to illustrate how COMAS works.

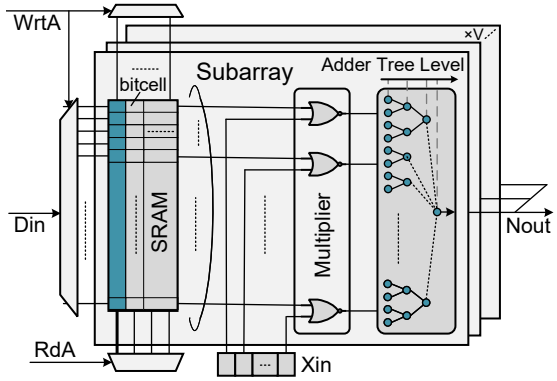



**Fig. 3.** Traditional CIM macro architecture.

$Col$, the logic gate functions as a bit-wise multiplier, and the adder tree is capable of performing 1-way-$Row$-in bit-serial summation [15]. For each subarray, $Din$ is written to a bitcell selected by the write address ($WrtA$). The read address ($RdA$) activates a whole column of the SRAM and conducts vector-vector multiplication with the input $Xin$. $WrtA$, $RdA$, and $Xin$ are broadcast to all subarrays, enabling the CIM macro to conduct GEMM.

Digital CIM macro presents numerous benefits, including high accuracy and low power consumption. However, previous works have primarily focused on GEMM-oriented approaches, rendering them incompatible with lightweight CNNs. In response to this demand, we propose *CAMO*, with a convolution-capable CIM macro to reduce activation traffic, and a configurable adder tree to improve MAC utilization of small-scale GEMM.

## III. A NOVAL CONVOLUTION MAPPING MECHANISM

To reduce activation traffic and improve MAC utilization of DWConv, we propose COMAS, including Ifmap mapping and weight reordering.

### A. Input Feature Map Mapping

Figure 2(c) shows the convolution procedure on the SRAM array for Ifmap presented in Figure 2(a), the row groups are denoted as $R_{0\sim2}$, and the columns are denoted as $C_{0\sim3}$, the horizontal axis represents the sliding windows. The computation process can be segmented into several critical steps as illustrated in Algorithm 1: (1) Initialize the SRAM with the first three rows of Ifmap, and each set of three columns of pixels corresponds to one column of the SRAM. (2) Activate pixels that correspond to the current slide window, e.g., at the first slide window, column $C_0$ and all row groups of the SRAM are activated, all pixels in the first column of the SRAM are read out. At the second slide window, two columns $C_0$ and $C_1$ of the SRAM will be activated, but only activate $R_1, R_2$ row groups, thus the 3rd to 8th rows of the first column, as well as the 0th to 2nd rows of the second column are read out, and meanwhile update 0th rows in the first column of the SRAM using the pixels from the fourth row of the Ifmap.

Additionally, Ifmaps need to be split into multiple $H \times L \times C$ blocks while $W > L$, as shown in Figure 2(e), bringing a small amount of redundant data, the activation traffic is thus as follows:

$$\begin{aligned} CAMO_{at} &= HWC + redundancy \\ &= HWC + \frac{(W - Col \times K)(K - S) \times HC}{Col \times K + S - K} \end{aligned} \tag{1}$$

While using the $im2col$ [14] method of baseline architectures, the activation traffic is:

$$Baseline_{at} = (\frac{W-K}{S}+1) \times (\frac{H-K}{S}+1) \times CK^2 \tag{2}$$

### B. Two-level Cyclic Shift Weight Reordering

To generate weight vectors that align with the pixel sequences, we design a two-level cyclic shift weight reordering strategy. We expand the convolution kernels in Figure 2(b) in a sliding window manner, as shown in Figure 2(d). Then, we observe two scenarios: (1) Kernels are sliding within the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:06 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1:** Ifmap mapping on CIM macro

**input :** Ifmap: $F_o[C][H][W]$
**output:** Pixels in slide window: $F_v$

**for** *each rc in* $0 \rightarrow Col$ **do**
  **for** *each c in* $0 \rightarrow C$ **do**
    **for** *each co in* $0 \rightarrow K$ **do**
      **for** *each ro in* $0 \rightarrow K$ **do**
        $SRAM[ro + 3co + 9c][rc] = F_o[c][ro][co + 3rc]$
**for** *each r in* $0 \rightarrow \frac{H-K}{S}$ **do**
  **for** *each i in* $0 \rightarrow Col - 1$ **do**
    **for** *each j in* $0 \rightarrow \frac{K}{S}$ **do**
      $F_v = \{SRAM[R_0][i + j/2],$
        $(SRAM[R_1][i + j//2], SRAM[R_2][i]\}$
      $SRAM[3j][i] = F_o[0][3 + r][3i + j]$

same row of Ifmap, e.g., in the 1st to 10th slide windows, weight vectors are cyclically shifted at the granularity of the kernel column within a kernel channel. (2) Kernels are switching between rows of Ifmap, e.g., in the 10th to 11th sliding windows, the weight vectors are cyclically shifted at the granularity of the kernel element within a kernel column. Therefore, the weight reordering algorithm employs a two-level cyclic shift, the outer loop is column-wise corresponding to the same row of Ifmap, and the inner loop is element-wise which occurs when there is a switch between rows of Ifmap.

## IV. CAMO Hardware Architecture

### A. Overall Architecture

Having a novel convolution mapping mechanism, COMAS, we next design the hardware architecture, *CAMO*. *CAMO* consists of a convolution-capable CIM macro (C-CIM), multiple weight reordering units (WRU), a master controller (MC), an adaptive weight duplication unit (AWDU), and a global buffer as shown in Figure 4. C-CIM contains a flexible SRAM array, and a configurable adder tree (CAT, Section IV-B). Each WRU is bound to one convolution input channel. MC generates the read/write address for C-CIM, and configurations for the CAT. AWDU is used to duplicate weights based on the size of the weight matrix adaptively (Section IV-C). The global buffer stores and reuses weight and output image data in convolution mode, and storing Ifmaps and the intermediate results in GEMM mode.

### B. Convolution-Capable CIM Macro

We customize a convolution-capable CIM macro based on [11], which supports both convolution and small-scale GEMM computing modes, we only made a few minor changes: (1) Enable to activate two SRAM columns while reading, with a row selection signal. (2) Increase the width of write data to 1024 bits, to support writing 4 rows simultaneously, with a configurable write address interval, e.g., 1 for initialization,

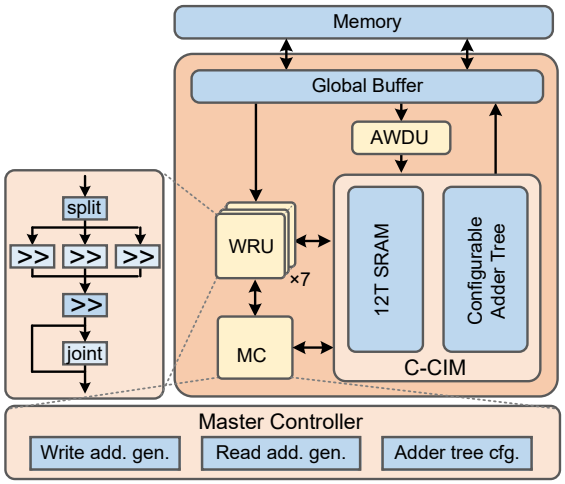



**Fig. 4.** CAMO hardware architecture.

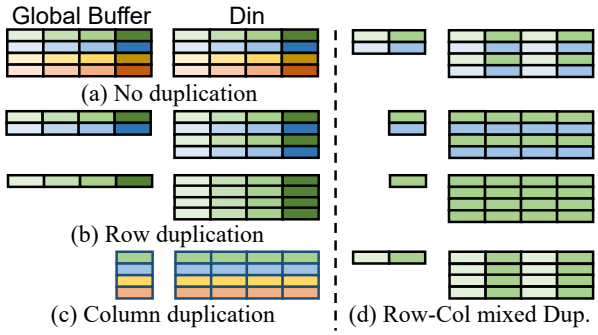



**Fig. 5.** Weight duplication.

9 for image updating. (3) Make adder tree configurable to support multiple compute mode, such as 1-way-64-in, 2-way-32-in, 4-way-16-in, 7-way-9-in, inspired by the adder tree proposed in [16]. (4) Increase the width of input data to 1024 bits, and every 256 bits are broadcast to 16 subarrays, to handle small-scale GEMM along with the CAT.

### C. Matrix-Scale-Aware Adaptive Weight Duplication Unit

In order to improve the MAC utilization of small-scale GEMM, we design a matrix-scale-aware adaptive weight duplication unit. It automatically reorganizes the duplication based on the size of the weight matrix when loading weights from the global buffer, as shown in Figure 5. There are 4 categories based on different duplication dimensions: no duplication, row duplication, column duplication, and row-column mixed duplication. Then, configure the write address interval parameter based on the number of rows duplicated. Note that the duplication process occurs when the weights are written from global buffer to C-CIM, thus not increasing memory overhead.

## V. Evaluation

### A. Experimental Settings

*1)* ***Workloads:*** We choose eight representative CNN workloads under ImageNet [17] to demostrate the benefits of *CAMO*, covering four traditional CNNs: ResNet34 (Res34), ResNet50 (Res50), VGG16, VGG19 and 4 lightweight

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:06 UTC from IEEE Xplore. Restrictions apply.

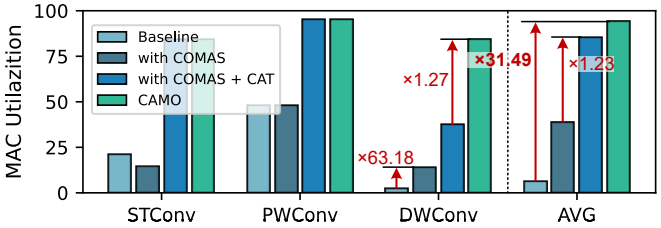



**Fig. 6.** The average multiplier utilization in processing STConv, DWConv, and PWConv of MobileNetV2.

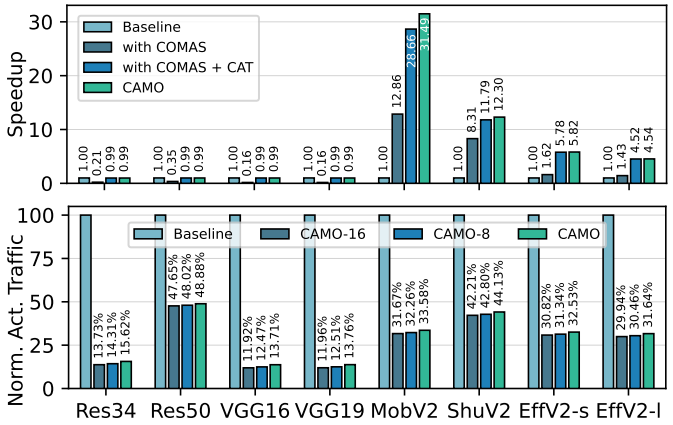



**Fig. 7.** Performance (upper) and traffic (bottom) comparison.

CNNs: MobileNetV2 (MobV2), ShuffleNetV2 (ShuV2), EfficientNetV2-s (EffV2-s), EfficientNetV2-l (EffV2-l).

*2) **Hardware modeling:*** We utilize 64 subarrays and a 512 KB global buffer in *CAMO*. The size of SRAM in each subarray is $64 \times 4$, each bitcell is 4 bits. The adder tree is configurable, and the bit widths of both $Din$ and $Xin$ are 1024. We implement the key components in RTL, design the layout of C-CIM, and synthesize them in the typical corner of 12-nm CMOS technology at 500 MHz to evaluate area and power consumption. Furthermore, we develop a simulator to access performance, traffic, and MAC utilization.

### B. Experimental Results

*1) **MAC utilization:*** To elucidate the high-performance gains of *CAMO*, we compare its MAC utilization with 3 counterparts in MobileNetV2: (1) *Baseline*, the traditional CIM architecture [11]. (2) *with COMAS*, baseline enhanced with COMAS. (3) *with COMAS + CAT*, baseline incorporates both COMAS and CAT. As illustrated in Figure 6, the integration of COMAS led to a substantial increase in MAC utilization for DWConv, while the addition of CAT further enhanced MAC utilization for PWConv. Consequently, *CAMO* achieved an overall MAC utilization of 92.3%, representing a **31.49**× improvement over the baseline.

*2) **Performance:*** Figure 7 (upper) shows the speedup of eight workloads. In traditional CNNs, *CAMO* results in a performance decline of less than 0.01%, while effectively eliminating the complex *im2col* process. In the case of lightweight CNNs, *CAMO* demonstrates significant performance improvements, ranging from 4.54× to 31.49×.

*3) **Activation traffic analysis:*** Figure 7 (bottom) compares the activation traffic of *CAMO* with a baseline adopting $im2col$ algorithm, to better understand the advantage of COMAS, we also include two hypothetical designs: CAMO-8 and CAMO-16, with the columns of the SRAM of C-CIM set to 8 and 16, respectively. Owing to COMAS, *CAMO* outperforms the baseline by **77.5%** activation traffic reduction on average. In addition, the activation traffic could be further reduced with the increasing of the columns of the SRAM of C-CIM.

TABLE I: Area and power breakdown of CAMO

| Components | Area ($\mu m^2$) | Power (mW) |
|---|---|---|
| C-CIM | 115200.0 ( 16.64%) | 18.16 ( 13.40%) |
| Global buffer | 568895.5 ( 82.17%) | 101.15 ( 74.78%) |
| 7 WRU | 5662.8 ( 0.82%) | 10.7 ( 7.91%) |
| AWDU | 2107.6 ( 0.30%) | 4.45 ( 3.29%) |
| MC | 443.6 ( 0.06%) | 0.80 ( 0.59%) |
| Total | 692309.5 (100.00%) | 135.26 (100.00%) |

TABLE II: Energy and area efficiency comparison

| | Pla.-Tec. (nm) | Freq. (MHz) | Perf. (GOPS) | Prec. (bit) | EE (TOPS/W) | AE (TOPS/mm$^2$) |
|---|---|---|---|---|---|---|
| [4] | FPGA-28 | 200 | 246.7 | 8 | 0.050 | - |
| [5] | FPGA-28 | 200 | 413.9 | 4 | 0.068 | - |
| [7] | ASIC-28 | 500 | 200 | 8 | 0.270 | 0.013 |
| **CAMO** | ASIC-12 | 1000 | 1890 | 4-8 | **13.976**-3.494 | **2.732**-0.683 |
| | scale† | 428.6 | 810 | | 2.567-0.642 | 0.215-0.054 |

† The experimental results are scaled from implement technology to 28-nm: $s = \frac{12}{28}, f_{28} = f_{12} * s, Area_{28} = \frac{Area_{12}}{s^2}, Power_{28} = \frac{Power_{12}}{s}$.

*4) **Area and power breakdown:*** Table I shows the area and power breakdown of *CAMO*. The chip area and power consumption of C-CIM only account for 16.64% and 13.4% of *CAMO*, respectively, and are dominated by the 512 KB global buffer, which is really small compared to the SOTA designs (e.g., 4 Mb *v.s.* 13.7 Mb [5]). While the remaining portions only occupy a small amount of area and energy expenditure.

*5) **Hardware efficiency comparison:*** Table II compares the hardware efficiency of *CAMO* with SOTA designs. The energy and area efficiencies of *CAMO* reach **13.976** TOPS/W and **2.732** TOPS/mm$^2$, respectively. To ensure a fair comparison, the experimental results under the 12-nm process have been scaled to the 28-nm process, using the method proposed in [18]. Based on the scaled results, the energy efficiency of *CAMO* is $12.83\times \sim 37.75\times$ higher than that of FPGA-based accelerators. Additionally, the energy and area efficiencies are $2.38\times$ and $4.27\times$ higher than those of ASIC-based accelerators, respectively.

## VI. Conclusion

In this work, we propose a CIM-based architecture, *CAMO*, which features a convolution-capable CIM macro with a convolution mapping mechanism. *CAMO* is further complemented by a configurable adder tree to efficiently handle small-scale GEMM. Our evaluation demonstrates that *CAMO* significantly outperforms the baseline up to 31.49× MAC utilization and 77.5% activation traffic reduction.

## Acknowledgment

This work was supported by National Natural Science Foundation of China (Grant No. U23A20301, 62102433), and NUDT Foundation (No. ZK2023-16, 2022-KJWPDL-08).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:06 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] M. Sandler, A. G. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 4510–4520.

[2] N. Ma, X. Zhang, H. Zheng, and J. Sun, "Shufflenet V2: practical guidelines for efficient CNN architecture design," in *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XIV*, ser. Lecture Notes in Computer Science, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds., vol. 11218. Springer, 2018, pp. 122–138. [Online]. Available: https://doi.org/10.1007/978-3-030-01264-9_8

[3] M. Tan and Q. V. Le, "Efficientnetv2: Smaller models and faster training," vol. 139, pp. 10 096–10 106, 2021. [Online]. Available: http://proceedings.mlr.press/v139/tan21a.html

[4] J. Li, J. Chen, K. Un, W. Yu, P. Mak, and R. P. Martins, "A 50.4 gops/w fpga-based mobilenetv2 accelerator using the double-layer MAC and DSP efficiency enhancement," in *IEEE Asian Solid-State Circuits Conference, A-SSCC 2021, Busan, Korea, Republic of, November 7-10, 2021*. IEEE, 2021, pp. 1–3. [Online]. Available: https://doi.org/10.1109/A-SSCC53895.2021.9634838

[5] L. Xuan, K. Un, C. Lam, and R. P. Martins, "An fpga-based energy-efficient reconfigurable depthwise separable convolution accelerator for image recognition," *IEEE Trans. Circuits Syst. II Express Briefs*, vol. 69, no. 10, pp. 4003–4007, 2022. [Online]. Available: https://doi.org/10.1109/TCSII.2022.3180553

[6] R. Xu, S. Ma, Y. Wang, X. Chen, and Y. Guo, "Configurable multi-directional systolic array architecture for convolutional neural networks," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 4, pp. 42:1–42:24, 2021. [Online]. Available: https://doi.org/10.1145/3460776

[7] S. Ryu, Y. Oh, and J. Kim, "Mobileware: A high-performance mobilenet accelerator with channel stationary dataflow," in *IEEE/ACM International Conference On Computer Aided Design, ICCAD 2021, Munich, Germany, November 1-4, 2021*. IEEE, 2021, pp. 1–9. [Online]. Available: https://doi.org/10.1109/ICCAD51958.2021.9643497

[8] Z. Zhang, J. Chen, X. Chen, A. Guo, B. Wang, T. Xiong, Y. Kong, X. Pu, S. He, X. Si, and J. Yang, "From macro to microarchitecture: reviews and trends of sram-based compute-in-memory circuits," *Sci. China Inf. Sci.*, vol. 66, no. 10, 2023. [Online]. Available: https://doi.org/10.1007/s11432-023-3800-9

[9] B. Wang, C. Xue, Z. Feng, Z. Zhang, H. Liu, L. Ren, X. Li, A. Yin, T. Xiong, Y. Xue, S. He, Y. Kong, Y. Zhou, A. Guo, X. Si, and J. Yang, "A 28nm horizontal-weight-shift and vertical-feature-shift-based separate-wl 6t-sram computation-in-memory unit-macro for edge depthwise neural-networks," in *IEEE International Solid-State Circuits Conference, ISSCC 2023, San Francisco, CA, USA, February 19-23, 2023*. IEEE, 2023, pp. 134–135. [Online]. Available: https://doi.org/10.1109/ISSCC42615.2023.10067526

[10] Y. Lo and R. Liu, "Morphable CIM: improving operation intensity and depthwise capability for SRAM-CIM architecture," in *60th ACM/IEEE Design Automation Conference, DAC 2023, San Francisco, CA, USA, July 9-13, 2023*. IEEE, 2023, pp. 1–6. [Online]. Available: https://doi.org/10.1109/DAC56929.2023.10247750

[11] H. Fujiwara, H. Mori, W. Zhao, M. Chuang, R. Naous, C. Chuang, T. Hashizume, D. Sun, C. Lee, K. Akarvardar, S. Adham, T. Chou, M. E. Sinangil, Y. Wang, Y. Chih, Y. Chen, H. Liao, and T. J. Chang, "A 5-nm 254-tops/w 221-tops/mm$^2$ fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE International Solid-State Circuits Conference, ISSCC 2022, San Francisco, CA, USA, February 20-26, 2022*. IEEE, 2022, pp. 1–3. [Online]. Available: https://doi.org/10.1109/ISSCC42614.2022.9731754

[12] H. Kim, T. Yoo, T. T. Kim, and B. Kim, "Colonnade: A reconfigurable sram-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid State Circuits*, vol. 56, no. 7, pp. 2221–2233, 2021. [Online]. Available: https://doi.org/10.1109/JSSC.2021.3061508

[13] B. Yan, J. Hsu, P. Yu, C. Lee, Y. Zhang *et al.*, "A 1.041-mb/mm$^2$ 27.38-tops/w signed-int8 dynamic-logic-based adc-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE International Solid-State Circuits Conference, ISSCC 2022, San Francisco, CA, USA, February 20-26, 2022*. IEEE, 2022, pp. 188–190.

[14] J. Yang, M. Wen, J. Shen, Y. Cao, M. Tang, R. Yang, J. Fei, and C. Zhang, "Bp-im2col: Implicit im2col supporting AI backpropagation on systolic arrays," pp. 415–418, 2022. [Online]. Available: https://doi.org/10.1109/ICCD56317.2022.00068

[15] P. Judd, J. Albericio, T. H. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 2016, pp. 19:1–19:12. [Online]. Available: https://doi.org/10.1109/MICRO.2016.7783722

[16] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *IEEE International Symposium on High Performance Computer Architecture, HPCA 2020, San Diego, CA, USA, February 22-26, 2020*. IEEE, 2020, pp. 58–70. [Online]. Available: https://doi.org/10.1109/HPCA47549.2020.00015

[17] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. S. Bernstein, A. C. Berg, and L. Fei-Fei, "Imagenet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015. [Online]. Available: https://doi.org/10.1007/s11263-015-0816-y

[18] Y. Wang, J. Lin, and Z. Wang, "An energy-efficient architecture for binary weight convolutional neural networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 2, pp. 280–293, 2018.


Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:06 UTC from IEEE Xplore. Restrictions apply.